

CTIA MediaT - Documentation

Une plateforme de gestion de documentation basée sur **Symfony 7.4 (LTS)** avec authentification, gestion de dossiers et de documents.

Table des matières

- [Installation](#)
 - [Démarrage](#)
 - [Tester avec Docker](#)
 - [Architecture du projet](#)
 - [Notes de version \(application\)](#)
 - [Mise à jour de Symfony](#)
 - [CI/CD et déploiement en production](#)
 - [Mise en production et workflow Git](#)
 - [Guide de développement](#)
 - [Bonnes pratiques de code](#)
 - [Mise en production \(manuel, hors pipeline\)](#)
 - [Gestion de la base de données](#)
-

Installation

Prérequis

- **PHP** >= 8.2 (recommandé : même version que le serveur et que le job CI, actuellement **8.2**)
- **Composer** 2.x
- **Docker** et **Docker Compose v2** (optionnel mais recommandé pour PostgreSQL et les e-mails de dev)
- **PostgreSQL** (ou autre SGBD supporté par Doctrine, via Docker ou installation locale)

Étapes d'installation

1. Cloner le projet

```
git clone <repository-url>
cd mediat
```

2. Installer les dépendances

```
composer install
```

3. Configurer les variables d'environnement

```
cp .env .env.local
```

Éditer `.env.local` et configurer notamment :

- `DATABASE_URL` — connexion à la base (voir [Tester avec Docker](#))
- `APP_SECRET` — clé secrète de l'application
- `MAILER_DSN` — en local avec Mailpit : `smtp://localhost:<port_smtp>` (voir plus bas)

4. Créer la base et appliquer les migrations

```
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate
```

Démarrage

Mode développement (PHP local)

```
# Serveur Symfony (recommandé)
symfony serve
# ou serveur PHP intégré
php -S localhost:8000 -t public/
```

L'application est accessible sur l'URL affichée par la commande (souvent `https://127.0.0.1:8000` avec `symfony serve`).

Tester avec Docker

Le dépôt inclut les fichiers `compose.yaml` et `compose.override.yaml` générés par Symfony Flex. Ils servent surtout à lancer **PostgreSQL** et **Mailpit** (capture des e-mails en développement) **sans** conteneur PHP : vous exécutez toujours l'app avec votre PHP local ou celui de l'IDE.

Démarrer les services

```
docker compose up -d
```

Services concernés :

Service	Rôle
<code>database</code>	PostgreSQL 16 (image Alpine par défaut)

Dans `compose.override.yaml`, le port **5432** du conteneur PostgreSQL est publié sur un **port aléatoire** de la machine hôte. Pour connaître le port réel :

```
docker compose ps
```

Exemple de `DATABASE_URL` (`.env.local`)

Adaptez l'hôte, le port exposé, l'utilisateur, le mot de passe et le nom de base (valeurs par défaut souvent `app` / `!ChangeMe!` selon `compose.yaml`) :

```
DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:<PORT_EXPOSED>/app?serverVersion=16&charset=utf8"
```

Ensuite : `php bin/console doctrine:database:create` (si besoin) et `php bin/console doctrine:migrations:migrate`.

Arrêt

```
docker compose down
```

Architecture du projet

```
mediat/
├── bin/                                # Scripts console
│   ├── console                        # Commandes Symfony
│   └── import-documents               # Import de documents
├── config/                            # Configuration de l'application
├── public/                            # Répertoire web (point d'entrée)
├── src/
│   ├── Controller/
│   ├── Entity/
│   ├── Form/
│   ├── Repository/
│   ├── Service/
│   └── ...
├── templates/
├── migrations/
├── compose.yaml                      # Stack Docker (PostgreSQL, etc.)
├── compose.override.yaml             # Ports de dev (DB, Mailpit)
├── composer.json
└── .env
```

Notes de version (application)

Les notes affichées dans l'interface (page **Notes de version**, lien sur le mot **Version** dans le pied de page) viennent du code, avec **un seul endroit** pour le numéro affiché dans le footer et la version « actuelle » de la page.

1. `config/services.yaml` — paramètre `app.version` (ex. `'1.3.0'`). Il alimente :
 - la variable globale Twig `app_version` (footers `base.html.twig` et `base_error.html.twig`),
 - la page notes de version (version sélectionnée par défaut et libellé « actuelle »).
2. `src/Controller/ReleaseNotesController.php` — méthode `releaseNotes()` : historique (plus récent en premier). Chaque entrée a :
 - `version` (semver),
 - `releasedAt` (`DateTimeImmutable`),
 - `items` (puces).

En **environnement de debug**, une exception est levée si `app.version` ne correspond pas à la `version` de la **première** entrée de `releaseNotes()` (pour éviter les oublis).

3. **Après une release** : mettre à jour `app.version`, ajouter une entrée en tête de `releaseNotes()` avec la même version, date et puces — **plus besoin de toucher aux templates de pied de page**.

Mise à jour de Symfony

Le projet cible la branche **7.4.*** définie dans `composer.json` (`extra.symfony.require` et contraintes `symfony/*`).

Support officiel (LTS 7.4)

Symfony publie le calendrier sur symfony.com/releases. Pour **Symfony 7.4** (version **Long Term Support**) : **PHP ≥ 8.2**, correctifs de **bugs** jusqu'à **fin novembre 2028**, correctifs de **sécurité** jusqu'à **fin novembre 2029** (vérifier toujours la page officielle pour les dates à jour).

Montée de version mineure / patch (reste en 7.4)

```
composer update "symfony/*"
```

Puis exécuter les tests et le lint localement (comme en CI).

Passage à une nouvelle version majeure (ex. 8.x)

1. Lire [UPGRADE-8.0.md](#) (ou la majeure cible) et la doc « [Upgrading a Major Version](#) ».
2. Vérifier la **version PHP** requise (Symfony 8.x nécessite **PHP 8.4+**).
3. Adapter `composer.json` (`extra.symfony.require` et toutes les contraintes `symfony/*`), puis `composer update`.
4. Corriger les dépréciations signalées en **7.4** avant de basculer, pour limiter les ruptures.

CI/CD et déploiement en production

Le dépôt est relié à **GitHub Actions**. Le workflow « **CI and deploy** » est défini dans `.github/workflows/deploy.yml`.

Comportement

Événement	Résultat
Pull request vers <code>main</code> ou <code>develop</code>	Job <code>test</code> uniquement (pas de déploiement)
Push sur <code>main</code> (merge inclus)	<code>test</code> puis <code>deploy</code>
Exécution manuelle (<code>workflow_dispatch</code>) sur <code>main</code>	<code>test</code> puis <code>deploy</code>

Job `test` (CI)

- Environnement : Ubuntu, **PHP 8.2**, SQLite pour les commandes Doctrine en CI.
- Étapes typiques : `composer install`, `composer validate --strict`, `lint:yaml`, `lint:twig`, `lint:container`.

Toute PR doit passer ces contrôles avant fusion.

Job `deploy` (CD)

- S'exécute **uniquement** sur la branche `main`, après un `push` ou un lancement manuel.
- Synchronise le code vers le serveur avec `rsync` (répertoires `bin/`, `config/`, `migrations/`, `src/`, `templates/`, ainsi que `composer.json` / `composer.lock`).
- Le répertoire `public/` n'est **pas** écrasé par `rsync` (uploads, fichiers serveur préservés).

Secrets GitHub requis

Dans **Settings** → **Secrets and variables** → **Actions** :

Secret	Description
<code>SSH_PRIVATE_KEY</code>	Clé privée SSH complète
<code>DEPLOY_HOST</code>	Hôte (IP ou FQDN)
<code>DEPLOY_USER</code>	Utilisateur SSH
<code>DEPLOY_PATH</code>	Chemin absolu du projet (ex. <code>/var/www/mediat</code>)

Optionnel : `DEPLOY_SSH_PORT` (sinon port 22).

Commandes exécutées sur le serveur après `rsync`

Le workflow lance à distance :

- `composer install --no-dev --no-interaction --optimize-autoloader --no-scripts`
- `php bin/console doctrine:migrations:migrate --no-interaction --env=prod`

- `php bin/console cache:clear --env=prod --no-debug --no-ansi`

Important : si de nouveaux bundles publient des assets dans `public/bundles/`, il peut être nécessaire sur le serveur (une fois) :

```
php bin/console assets:install public --env=prod
```

Les variables d'environnement de production (`APP_ENV=prod`, `DATABASE_URL`, `APP_SECRET`, etc.) restent à configurer **sur le serveur** (fichiers `.env.local`, variables système ou secrets Symfony) : elles ne sont pas déposées par le workflow.

Mise en production et workflow Git

Passer par la pipeline une fois le setup terminé

Après la **première installation** du projet sur le serveur et la configuration des **secrets GitHub Actions** (voir ci-dessus), il est **fortement recommandé de ne plus déployer « à la main »** sur la production : faites transiter **toutes** les mises en production par la **pipeline** (merge sur `main` → job `test` puis `deploy`). Cela garantit que les tests passent, que les migrations et le vidage de cache sont exécutés de la même façon à chaque fois, et limite les oublis. La section [Mise en production \(manuel, hors pipeline\)](#) reste utile pour un **dépannage ponctuel** ou une **initialisation** du serveur, pas pour le déploiement courant.

Travailler proprement avec Git

1. **Se placer sur la bonne branche** (branche de fonctionnalité, `develop`, etc. — selon les habitudes de l'équipe, jamais le travail « au hasard » sur une branche qui n'est pas la vôtre) :

```
git checkout <nom-de-branche>
```

2. **Récupérer les derniers changements du dépôt distant** avant de modifier ou de committer :

```
git pull
```

3. **Vérifier ce qui a changé :**

```
git status
```

4. **Indexer uniquement les fichiers que vous voulez inclure dans le commit.** Préférez des chemins explicites plutôt que tout ajouter d'un coup :

```
git add chemin/vers/fichier1 chemin/vers/fichier2
```

Évitez `git add .` en routine : vous risqueriez de versionner par erreur des fichiers locaux (`.env.local`, caches, artefacts, fichiers personnels) qui ne doivent pas aller sur le dépôt.

5. **Créer un commit avec un message clair** (ce que fait le changement, pourquoi — suffisamment pour comprendre l'historique plus tard) :

```
git commit -m "Description courte et significative du changement"
```

6. **Pousser votre branche :**

```
git push
```

Mettre en production (merge vers `main`)

Quand la fonctionnalité est prête et validée en revue : **fusionnez vers `main`** (souvent via une **pull request** sur GitHub, puis merge une fois les contrôles verts). C'est le **push / merge sur `main`** qui déclenche le **déploiement** automatique vers le serveur (voir [Comportement](#) du workflow).

Vérifier sur GitHub après le déploiement

Sur le dépôt GitHub : onglet **Actions**, ouvrez l'exécution correspondant à votre merge. Vérifiez que le job **test** et le job **deploy** se terminent **avec succès**. Tant que la pipeline est en échec ou incomplète, considérez que la production **n'est pas** fiabilisée par ce déploiement : corrigez le problème, poussez un nouveau commit ou relancez le workflow si besoin.

Guide de développement

Créer un nouveau contrôleur

```
php bin/console make:controller MonController
```

Exemple de contrôleur :

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

#[Route('/mon-chemin')]
class MonController extends AbstractController
```

```
{
    #[Route('/', name: 'app_mon_index')]
    public function index(): Response
    {
        return $this->render('mon/index.html.twig');
    }
}
```

Créer une entité

```
php bin/console make:entity MonEntite
```

Créer un formulaire

```
php bin/console make:form MonFormType
```

Générer une migration

Après modification d'une entité :

```
php bin/console doctrine:migrations:diff
php bin/console doctrine:migrations:migrate
```

Gestion de l'authentification

L'application utilise Symfony Security avec une authentification par email/mot de passe.

Entity User ([src/Entity/User.php](#)) :

- Email unique
- Mot de passe hashé (bcrypt)
- Rôles (ROLE_USER, ROLE_ADMIN, ROLE_CTIA)
- Sessions persistantes sur 2 jours avec système "Remember me"

Configuration ([config/packages/security.yaml](#)) :

- Firewall protégé par formulaire
- Logout sécurisé
- Contrôles d'accès par rôle

Gestion des uploads

Le service [FileManager](#) gère les uploads de documents :

- Validation du type de fichier

- Stockage sécurisé dans `public/uploads/documents/`
- Suppression des fichiers orphelins

Création et gestion des rôles

Dans Symfony, un **rôle** est une simple chaîne (convention : préfixe `ROLE_`, en majuscules avec underscores, ex. `ROLE_CTIA`). Ce n'est pas une table en base : les rôles « métier » d'un utilisateur sont stockés dans la colonne JSON `roles` de l'entité `User` et lus à la connexion pour les contrôles d'accès.

Ce projet utilise déjà trois rôles définis comme constantes dans `src/Entity/User.php` :

Constante	Valeur	Usage typique dans MediaT
<code>ROLE_USER</code>	<code>ROLE_USER</code>	Tout utilisateur validé, ajouté automatiquement par <code>getRoles()</code> même si la base ne le répète pas.
<code>ROLE_CTIA</code>	<code>ROLE_CTIA</code>	Profil « client CTIA » : peut être affecté aux utilisateurs et utilisé pour restreindre l'accès à certains dossiers .
<code>ROLE_ADMIN</code>	<code>ROLE_ADMIN</code>	Accès à tout le préfixe <code>/admin</code> (contrôleur protégé par <code># [IsGranted('ROLE_ADMIN')]</code>) et choix dans les restrictions de dossiers.

Ajouter un rôle équivalent pour un autre client (ex. `ROLE_MUTAMI`)

1. **Déclarer la constante** dans `src/Entity/User.php` à côté des autres :

```
final public const ROLE_MUTAMI = 'ROLE_MUTAMI';
```

Le nom PHP (`ROLE_PARTENAIRE_X`) et la valeur chaîne (`'ROLE_PARTENAIRE_X'`) suivent la même convention Symfony.

2. **Permettre de l'assigner à la création / édition d'un utilisateur** : ajouter une ligne dans les `choices` du champ `role` dans :

- `src/Form/UserFormType.php`
- `src/Form/EditUserFormType.php`
Libellé affiché à gauche, valeur du rôle à droite (comme pour « CTIA » → `User::ROLE_CTIA`). À la soumission, un seul rôle « principal » est enregistré dans `$user->setRoles([$role])`, `getRoles()` ajoute toujours `ROLE_USER` pour la compatibilité Symfony.

Une fois fait, il faut penser à lancer une migration de la base de données et un nettoyage du cache.

Les administrateurs attribuent les rôles via l'interface d'administration. Aucune migration Doctrine n'est nécessaire tant que vous ne changez pas la structure de l'entité (la colonne `roles` est déjà un tableau JSON).

Bonnes pratiques de code

Résumé orienté **Symfony 7** et ce dépôt. Pour le détail de chaque mécanisme, la [documentation Symfony](#) reste la référence.

1. Nommage et conventions

Les conventions suivent surtout **PHP** et **PSR-4** (arborescence `src/` ↔ namespace `App\`) :

- **Classes / fichiers** : PascalCase, le fichier porte le même nom que la classe (`User.php` → `class User`).
- **Méthodes et propriétés** : camelCase (`getUserEmail()`, `documentTitle`).
- **Constantes** : `UPPER_SNAKE_CASE` (`ROLE_ADMIN`, tailles maximales).
- **Noms de routes** : dans MediaT, préfixe `app_` + module + action (`app_document_view`), pour les retrouver avec `php bin/console debug:router`.

2. Structure des contrôleurs

Le **contrôleur** est le point d'entrée HTTP : il lit la requête, délègue au métier (services, repositories) et renvoie une **Response** (souvent du HTML via `$this->render()`).

- **AbstractController** fournit des aides : rendu Twig, redirections, JSON, `$this->getUser()`, levées d'exceptions HTTP, etc.
- **#[Route]** sur la classe et/ou les méthodes définit l'URL et le **nom de route** utilisé dans Twig avec `path('nom_route')`. Le nom final peut combiner préfixe de classe et suffixe de méthode : en cas de doute, utilisez `debug:router`.
- **Constructeur avec types typés** : grâce à **autowire**, Symfony injecte les dépendances sans configuration fichier pour la plupart des classes sous `App\`.

```
#[Route('/documents', name: 'app_document')]
class DocumentController extends AbstractController
{
    // Injection automatique : une instance par requête
    public function __construct(
        private DocumentRepository $repository,
        private FileManager $fileManager
    ) {}

    #[Route('/', name: 'index')]
    public function index(Request $request): Response
    {
        // Traitement métier, puis passage de variables au template
        return $this->render('document/index.html.twig', [
            // 'items' => $this->repository->findBy(...),
        ]);
    }
}
```

3. Utilisation des services

Placez la logique réutilisable dans `src/Service/`. Dans `config/services.yaml`, `App\` : enregistre les classes de `src/` comme services, **autowire** et **autoconfigure** activent l'injection et certaines intégrations (commandes, événements, etc.).

```
<?php

namespace App\Service;

class MonService
{
    public function __construct(
        private DocumentRepository $repository
    ) {}

    public function faireQuelquechose(): array
    {
        return $this->repository->findAll();
    }
}
```

Pour utiliser ce service, ajoutez **MonService** **\$monService** au constructeur d'un contrôleur ou d'un autre service : Symfony résout le câblage tout seul.

4. Validation des données

Sur un formulaire, les **constraints** (classes ****Assert****) sont évaluées lorsque vous appelez **\$form->handleRequest(\$request)** puis **\$form->isValid()**. Les erreurs sont alors disponibles dans le formulaire et dans Twig (**form_errors**, classes CSS sur les champs).

```
use Symfony\Component\Validator\Constraints as Assert;

class MonFormType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('email', EmailType::class, [
                'constraints' => [
                    new Assert\NotBlank(),
                    new Assert\Email(),
                ],
            ])
            ->add('password', PasswordType::class, [
                'constraints' => [
                    new Assert\NotBlank(),
                    new Assert\Length(['min' => 8]),
                ],
            ]);
    }
}
```

Les mêmes contraintes peuvent aussi être posées sur l'**entité** pour valider des données hors formulaire.

5. Requêtes Doctrine optimisées

Le piège **N+1** : après un `findAll()`, chaque accès à une association non chargée (ex. `$doc->getFolder()`) peut déclencher **une requête SQL supplémentaire**. Pour un listing, utilisez un **JOIN** et `addSelect('alias')` sur l'association pour tout charger en une fois (**fetch join**).

```
// Exemple : une seule requête qui charge les documents et leurs dossiers (évite le N+1)
public function findAllOrderedWithFolder(): array
{
    return $this->createQueryBuilder('d')
        ->leftJoin('d.folder', 'f')
        ->addSelect('f')
        ->orderBy('d.position', 'ASC')
        ->getQuery()
        ->getResult();
}

// Exemple de motif à éviter sur de gros volumes : lazy loading dans une boucle
public function exempleProblemeNPlus1(): void
{
    $documents = $this->findAll();
    foreach ($documents as $doc) {
        echo $doc->getFolder()?->getName(); // risque : 1 requête par document si
        folder non joint
    }
}
```

6. Sécurité dans les templates

En Twig, `{{ variable }}` échappe le HTML par défaut (**protection XSS**). Le filtre `|raw` affiche la chaîne telle quelle : à réserver au contenu **de confiance** (HTML généré et contrôlé par vous).

`is_granted('ROLE_...')` interroge les rôles de l'utilisateur connecté. On peut aussi tester des **permissions métier** (`is_granted('EDIT', object)`) si le projet définit des **Voters** pour cet attribut. Dans MediaT, les contrôles courants passent surtout par `ROLE_USER`, `ROLE_CTIA`, `ROLE_ADMIN` et les restrictions sur les dossiers.

```
{# Échappement par défaut – préféré pour tout texte utilisateur #}
<p>{{ document.title }}</p>

{# Dangereux si la source n'est pas sûre #}
<p>{{ document.content|raw }}</p>

{# Masquer un lien si l'utilisateur n'est pas admin #}
{% if is_granted('ROLE_ADMIN') %}
    <a href="{{ path('app_admin') }}">Administration</a>
{% endif %}
```

7. Gestion des erreurs

`createNotFoundException()` et `createAccessDeniedException()` produisent des réponses **404** et **403** adaptées. Si l'utilisateur n'est pas connecté alors que la route exige un rôle, le **firewall** peut **rediriger vers le login** avant même d'atteindre votre code.

```
if (!$document) {
    throw $this->createNotFoundException('Document non trouvé');
}

if (!$this->isGranted('ROLE_ADMIN')) {
    throw $this->createAccessDeniedException();
}
```

Pour une route ou un contrôleur entier, `#[IsGranted('ROLE_ADMIN')]` (attribut PHP sur la classe ou la méthode) centralise souvent la vérification.

8. Commentaires et documentation

- **PHPDoc** sur les méthodes publiques : paramètres, valeur de retour, exceptions éventuelles.
- Dans le corps du code, commenter surtout le **pourquoi** (règle métier, logique de récupération...).

```
/**
 * Récupère les documents accessibles par l'utilisateur.
 *
 * @param User $user Utilisateur connecté (filtrage par rôles / dossiers selon la
couche appelante)
 * @param int $limit Plafond du nombre de résultats
 *
 * @return array<Document> Liste triée par date décroissante
 */
public function findAccessibleDocuments(User $user, int $limit = 50): array
{
    // ...
}
```

9. Tests

Le **MakerBundle** génère une classe de test. **PHPUnit** (voir `phpunit.xml.dist`) exécute la suite. Les tests **fonctionnels** utilisent souvent **WebTestCase** pour simuler des requêtes HTTP contre le kernel Symfony.

```
php bin/console make:test DocumentControllerTest
php bin/phpunit
```

Mise en production (manuel, hors pipeline)

En routine, le déploiement passe par **GitHub Actions** (voir [CI/CD et déploiement en production](#)). Cette section reste utile pour une **première installation** ou un **dépannage** sur le serveur.

Important : vider le cache après modification

Après toute modification en production, vous **DEVEZ** vider le cache Symfony (le workflow le fait déjà après déploiement) :

```
php bin/console cache:clear --env=prod
```

Cette commande est **critique** pour :

- Recharger les nouveaux fichiers de configuration
- Mettre à jour les routes
- Appliquer les modifications de code

Checklist manuelle (serveur)

1. **Vérifier les configuration**

```
php bin/console config:dump
```

2. **Exécuter les migrations**

```
php bin/console doctrine:migrations:migrate --env=prod
```

3. **Installer les assets publics si nécessaire**

```
php bin/console assets:install --env=prod
```

4. **Vider le cache**

```
php bin/console cache:clear --env=prod
```

5. **Vérifier les logs**

```
tail -f var/log/prod.log
```

Variables d'environnement production

À configurer sur le serveur (par exemple `.env.prod.local` ou variables d'environnement du système) :

```
APP_ENV=prod
APP_DEBUG=0
DATABASE_URL="postgresql://user:password@host:5432/mediat?
serverVersion=16&charset=utf8"
APP_SECRET=your_secret_key_here
```

Configuration Nginx/Apache

Assurez-vous que le document root pointe vers `public/`.

Nginx :

```
server {
    listen 80;
    server_name mediat.example.com;
    root /path/to/mediat/public;

    location / {
        try_files $uri /index.php$is_args$args;
    }

    location ~ ^/index\.php(/|$) {
        fastcgi_pass php-fpm;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
```

Gestion de la base de données

Créer une migration

```
php bin/console doctrine:migrations:diff
php bin/console doctrine:migrations:migrate
```

Voir le statut des migrations

```
php bin/console doctrine:migrations:status
```

Rollback d'une migration

```
php bin/console doctrine:migrations:execute --down Version20251125122352
```

Rafraîchir la base de données (développement uniquement)

```
php bin/console doctrine:database:drop --force  
php bin/console doctrine:database:create  
php bin/console doctrine:migrations:migrate
```

Ressources utiles

- [Calendrier et support des versions Symfony](#)
- [Documentation Symfony 7.4](#)
- [Documentation Doctrine ORM](#)
- [Bonnes pratiques Symfony](#)

Projet développé et maintenu par Jules BOSSIS--GUYON. Pour toute question ou demandes d'évolutions, ouvrir une issue ou discussion. Ce répertoire reste à jour et les demandes seront étudiés.

Dernière mise à jour du README : mai 2026